BIG DATA ANALYTICS — CA3 ACTIVITY 2

Bengaluru Urban Mobility & Transit Analytics

Multi-modal (Bike · Auto · Cab) demand, congestion, fare and ETA analytics with MapReduce and machine learning

247,951	4	2	10
Trips analysed	MapReduce jobs	ML models	City hubs

Team Members: [Your Name], [Teammate Name] | MCA AIML — III Semester | [Date]

WHY THIS PROJECT

The real-world problem

Bengaluru riders choose between bike taxis, autos and cabs on roads that slow to a crawl at peak. Riders, drivers and city planners each need different answers from the same trip data.

1 When and where is demand highest?

By hour, hub and vehicle type.

2 Which ride should I take?

ETA and fare for bike vs auto vs cab.

3 Where are the choke points?

Slowest peak-hour corridors.

4 How do peaks and rain change trips?

Speed, delay and surge.

5 Where should drivers wait?

Fare earned per minute, by hour and corridor.

6 Can ML predict ETA and demand?

Regression for ETA, classification for surge.

STAGE 0

Dataset: synthetic Bengaluru trips

- No open trip-level dataset exists for Bengaluru bike / auto / cab rides, so trips are generated from documented city rules (seed 42, fully reproducible).
- 10 hubs with real coordinates, from Koramangala and Whitefield to Kempegowda Airport and Majestic.
- Traffic peaks 08:30–11:30 (1.5–2.2×) and 17:30–21:30 (1.6–2.4×); free-flow at night; rain adds 15%.
- Rate cards: Bike ₹20 + ₹8/km, Auto ₹30 + ₹15/km, Cab ₹60 + ₹18–22/km, plus delay charge, night surcharge and surge.
- 1 Jan – 30 Jun 2025 (181 days, 25 rainy). About 1.5% dirty rows were injected on purpose for the cleaning stage.

250,750
Raw rows

247,951
After cleaning

11
Raw columns

30
Columns after feature engineering

HOW IT ALL FITS TOGETHER

End-to-end pipeline



`pipeline/run_all.py` runs steps 0–4 in about 10 seconds. **`backend/app.py`** serves /api/dashboard?role=, /api/predict/trip and /api/predict/demand.

STAGE 1

Data cleaning & feature engineering

Cleaning step	Rows
Duplicate rows removed	**750**
Missing pickup zone removed	**600**
Zero / negative distance removed	**600**
Duration ≠ timestamps removed	**600**
Outliers > 99.9th pct (185 min) removed	**249**
Missing fares imputed from rate card	**600**
Vehicle names normalised (case / spaces)	**600**

Engineered features

- hour, day_of_week, is_weekend, month
- is_peak_hour, is_night, time_period
- traffic_factor, traffic_level
- free_flow_min, delay_min, avg_speed_kmph
- corridor (pickup → drop-off)
- distance_category, fare_per_km, fare_per_min

STAGE 2 — HADOOP-STREAMING COMPATIBLE

MapReduce implementation

Job 1 · Modal demand by hour

Map (hour | vehicle) → 1

Reduce sum → 72 rows

Job 2 · Duration & fare by zone

Map (zone | vehicle) → minutes, fare

Reduce count, averages, total → 30 rows

Job 3 · Congestion corridors

Map peak, inter-zone only: (pickup | drop) → km, min, delay

Reduce speed, delay share, status → 90 rows

Job 4 · Demand dataset

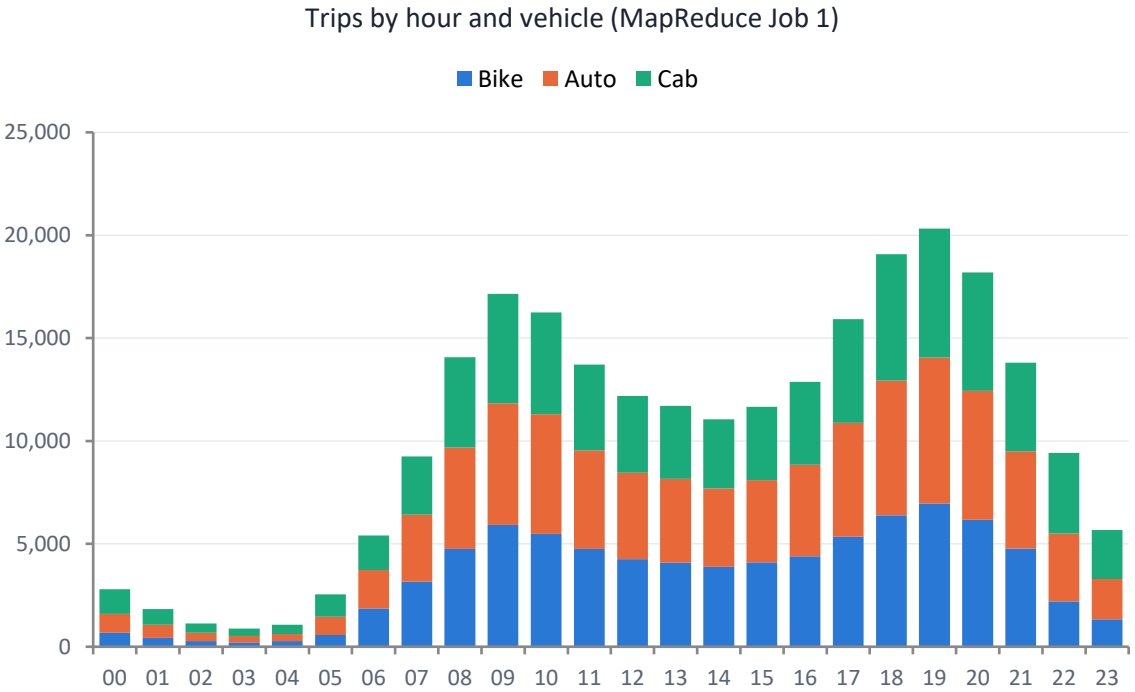
Map (date | hour | day | zone | rain) → 1

Reduce count → 38,158 rows → feeds Model B

Each job is a real mapper.py / reducer.py pair run as Map → Shuffle/Sort → Reduce subprocesses, the contract Hadoop Streaming uses. 247,951 records through all 4 jobs in 1.5 s locally.

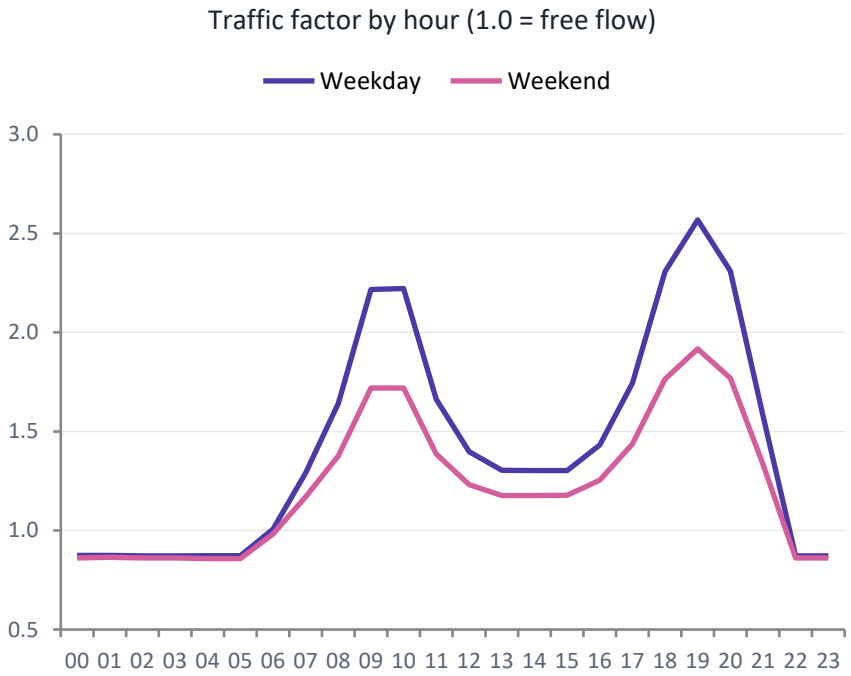
STAGE 3 — ANALYTICS

Demand peaks at 09:00 and 19:00



- Busiest hour 19:00, about 20,000 trips over the period.
- 48% of trips fall in the two peak windows.
- Mode share is balanced: bike 33%, auto 35%, cab 32%.
- Weekdays carry 76% of trips; Wednesday is busiest.

Choke points: two corridor clusters



Slowest peak-hour corridors (MapReduce Job 3)

Corridor	km/h	Delay
Koramangala → HSR Layout	**13.1**	10 min
Electronic City → HSR Layout	**13.1**	23 min
Majestic → MG Road	**13.2**	10 min
HSR Layout → Koramangala	**13.2**	10 min
MG Road → Majestic	**13.3**	10 min
Koramangala → Jayanagar	**13.4**	9 min

Of 90 corridors: 21 Critical (< 13.75 km/h), 46 Severe (< 15), 23 Moderate. Peak traffic factor reaches 2.6× at 19:00 on weekdays.

STAGE 3 — MODES

Bike vs auto vs cab

Bike

₹78
average fare

33% / 12%
of trips / of revenue

23 → 16.8
km/h, off-peak → peak

+47%
minutes per km in rain

Auto

₹172
average fare

35% / 28%
of trips / of revenue

19.5 → 12.8
km/h, off-peak → peak

+29%
minutes per km in rain

Cab

₹408
average fare

32% / 61%
of trips / of revenue

23.3 → 13.1
km/h, off-peak → peak

+31%
minutes per km in rain

WHAT THE DATA SHOWS

Key findings

19:00 Peak demand hour	**Koramangala** Busiest pickup hub	**17.4 km/h** Average trip speed
₹5.36 Cr Total fare value, 6 months	**1.39×** Average surge multiplier	**0.85** Distance ↔ duration correlation

- Drivers earn less per trip-minute in peak jams (auto ₹6.5/min by day vs ₹11.8/min at night): delay charges don't keep pace with time lost.
- At night an auto costs as much as a cab for trips of 8 km or more (₹184 vs ₹180 at 23:00), because of the 1.5× auto night meter.

STAGE 4

Machine learning models

MODEL A — REGRESSION

Trip duration (ETA)

Random Forest regressor · 80 trees, depth 12

- Features: vehicle, distance, hour, day, traffic factor, rain
- Learns actual ÷ formula ETA, so long trips still scale by vehicle
- Fare is then computed from the ETA via the rate card

2.995 min
Mean absolute error

0.957
R^2 score

MODEL B — CLASSIFICATION

Zone demand level → surge

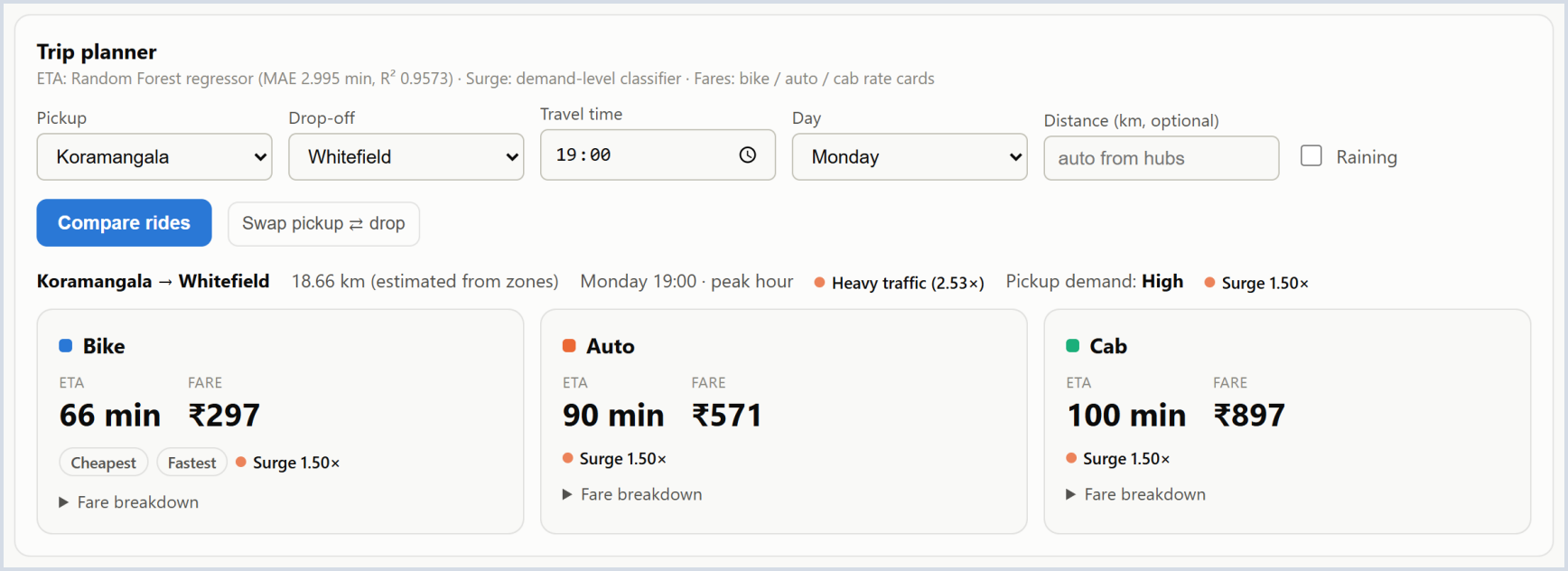
Random Forest classifier · Low / Medium / High

- Trained on 38,158 MapReduce Job 4 cells
- Features: hour, day, weekend, peak, zone, rain
- Tertile labels: Low ≤ 4, Medium ≤ 8, High > 8 trips / hour

68.0%
Accuracy (random = 33%)

0.687
Weighted F1

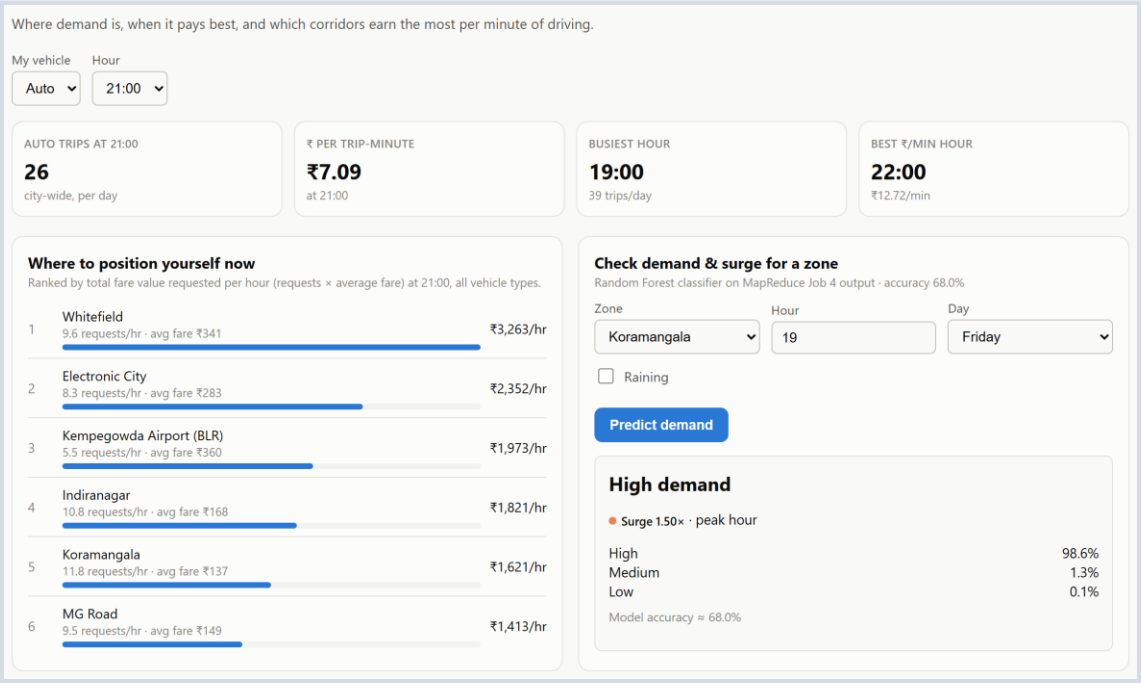
Trip planner: compare all three rides



Koramangala → Whitefield, Monday 19:00: bike 66 min / ₹297, auto 90 min / ₹571, cab 100 min / ₹897. ETA from Model A, surge from Model B, fares from the rate card.

THE DASHBOARD — DRIVER VIEW

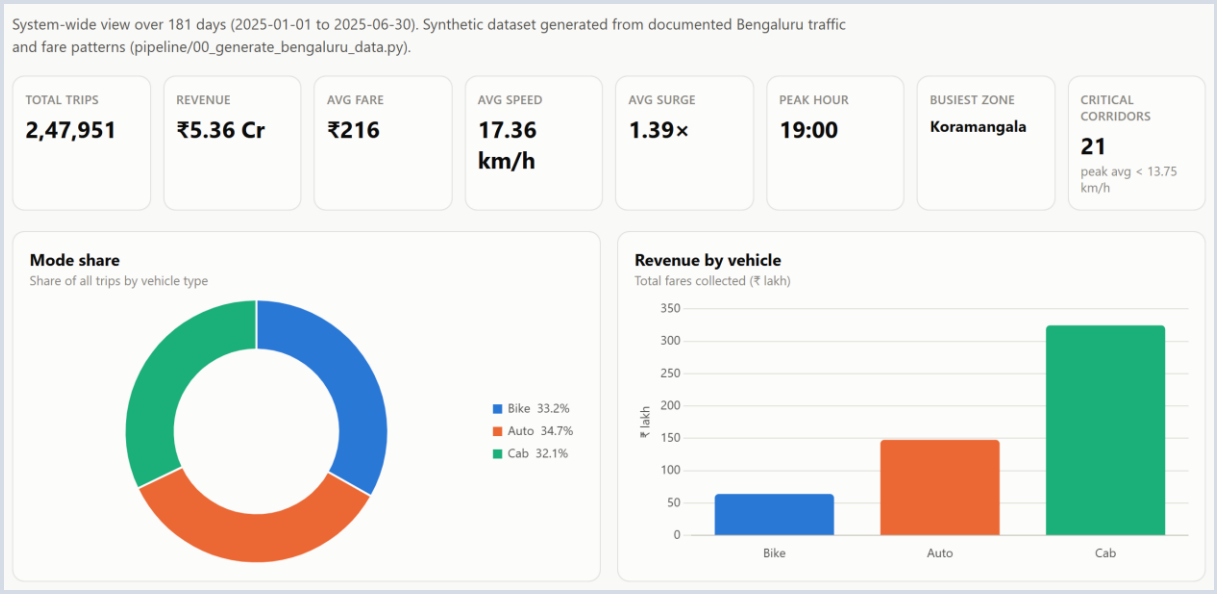
Where to wait, and when it pays



- Hubs ranked by fare value requested per hour at the chosen time.
- Demand & surge check calls Model B live (Koramangala, 19:00 → High, 1.5× surge).
- Also: zone × hour heatmap, earning-window chart, best-earning corridors.

THE DASHBOARD — ADMIN VIEW

System-wide view of all MapReduce outputs



- KPIs for the full 181 days: trips, revenue, speed, surge, critical corridors.
- Charts for all four MapReduce jobs, congestion, speed, rain impact and a day × hour heatmap.
- ML card shows both models' metrics next to the formula-only baseline.

SO WHAT?

Insights & recommendations

1	**Put supply where the peaks are**	Shift drivers to 09:00–10:00 and 18:00–20:00; Job 1 shows these hours carry the most trips.
2	**Fix two corridor clusters first**	Koramangala / HSR / Electronic City (Silk Board area) and Majestic ↔ MG Road are the slowest, at 13.1–13.3 km/h.
3	**Recommend by context**	Bikes for short peak trips (16.8 km/h vs 13.1 for cabs); cabs in rain (bikes +47% min/km).
4	**Review peak-hour driver pay**	₹/trip-minute is lowest in jams; a higher delay rate would keep drivers on the road.
5	**Revisit the night auto surcharge**	At 1.5× an auto costs as much as a cab for 8 km+ trips after 22:00.

BEING HONEST

Limitations & next steps

Synthetic data

Trips are generated from documented rules, not collected. Real operator or city data would validate the patterns.

ML ≈ formula

On this data the formula alone scores MAE 2.99 min, matching the model. Real data would show what ML adds.

Hadoop simulated

Jobs run locally with the Hadoop Streaming contract; the same mapper/reducer files can run on a real cluster.

Demand model at 68%

Weather forecasts, events and holidays would sharpen the Low / Medium / High split.

Thank you

A complete Big Data pipeline — generation, cleaning, MapReduce, analytics and machine learning — processing 247,951 Bengaluru trips end-to-end in about 10 seconds.

4	20+	2	3
MapReduce jobs	Charts & tables	ML models	Role views

Questions?